```
    public void setFirstName(String firstName) {
        mFirstName = firstName;
        notifyPropertyChanged(in.samvidinfotech.bindingdemo.
BR.firstName);
    }

    public String getLastName() {
        return mLastName;
    }

    public void setLastName(String lastName) {
        mLastName = lastName;
        notifyPropertyChanged(in.samvidinfotech.bindingdemo.
BR.lastName);
    }
}
```

The *BaseObservable* class makes our model observable for data changes. But the data binding class cannot just magically know when the data in the model changes; so we need to inform the framework when the data changes. To do that, we need to add the *@Bindable* annotation to the fields that we want observed. So whenever that property changes, we notify the framework with the convenience method *notifyPropertyChanged()*, which the *BaseObservable* class gives us. We pass the field, which is changed as argument in call to *notifyPropertyChanged()*. Here, we have a *BR* class, which contains constants of fields that are observable inside the whole application. This *BR* class is similar to the *R.java* class that keeps all of our resources' IDs. So our variables are observables; we have our observable fields as constants in the *BR* class. Note that if we use data binding, we can update our data from any thread, and data binding will take care of updating the data from the UI thread when the next frame comes.

Now, if you have your models extending from any other base class, or if for any other reason you can't extend the *BaseObservable* class, then you can implement an observable interface directly. The *BaseObservable* class itself implements *Observable* internally. So, if we switch to *Observable* in our UserData model, it would look something like what's shown below:

```
public class UserData implements Observable {
    private PropertyChangeRegistry mPropertyChangeRegistry;
    private String mFirstName;
    private String mLastName;

    public UserData(String firstName, String lastName) {
        mFirstName = firstName;
        mLastName = lastName;
        mPropertyChangeRegistry = new
PropertyChangeRegistry();
    }

    public void setFirstName(String firstName) {
        mFirstName = firstName;
        mPropertyChangeRegistry.notifyChange(this,
in.samvidinfotech.bindingdemo.BR.firstName);
    }

    public void setLastName(String lastName) {
        mLastName = lastName;
        mPropertyChangeRegistry.notifyChange(this,
in.samvidinfotech.bindingdemo.BR.lastName);
    }

    @Bindable
    public String getFirstName() {
        return mFirstName;
    }

    @Bindable
    public String getLastName() {
        return mLastName;
    }

    @Override
    public void addOnPropertyChangedCallback(OnPropertyChanged
Callback onPropertyChangedCallback) {
        mPropertyChangeRegistry.
add(onPropertyChangedCallback);
    }

    @Override
    public void removeOnPropertyChangedCallback(OnPropertyChan
gedCallback onPropertyChangedCallback) {
        mPropertyChangeRegistry.remove(onPropertyChangedCallb
ack);
    }
}
```

Here, we need to create an object of *PropertyChange Registry*, and we need to add and remove callbacks from it using the methods *addOnPropertyChangedCallback* and *removeOnPropertyChangedCallback*, respectively, which the observable interface gives us. You may notice that rather than adding the *@Bindable* annotation to variables, we have added it to getter methods, which is also a way of making our fields observable.

Working with lists

When it comes to working with lists, data binding is extremely helpful, while also being very easy to integrate. You can easily create a layout file, as shown earlier, for a list item. Then, in the adapter for your *RecyclerView*, you can store the binding in your *ViewHolder*, and in *onBindViewHolder()*, you just set the variable in binding. The code for *ViewHolder* is pretty small and neat.